CogExplore: Contextual Exploration with Language-Encoded Environment Representations

Harel Biggie, Patrick Cooper, Doncey Albin, Miles Mena, Christoffer Heckman

***Abstract*—Integrating language models into robotic exploration frameworks improves performance in unmapped environments by providing the ability to reason over semantic groundings, contextual cues, and temporal states. The proposed method employs large language models (GPT-3.5 and Claude Haiku) to reason over these cues and express that reasoning in terms of natural language, which can be used to inform future states. We are motivated by the context of search-and-rescue applications where efficient exploration is critical. We find that by leveraging natural language, semantics, and tracking temporal states, the proposed method greatly reduces exploration path distance and further exposes the need for environment-dependent heuristics. Moreover, the method is highly robust to a variety of environments and noisy vision detections, as shown with a 100% success rate in a series of comprehensive experiments across three different environments conducted in a custom simulation pipeline operating in Unreal Engine.**

***Index Terms*—Exploration, Natural Language, Contextual Reasoning**

I. Introduction

Exploring unmapped environments is paramount to modern robotic applications, such as search-and-rescue and assistive robotics. While state-of-the-art methods have made incredible progress in exploring in challenging search and rescue scenarios [1], [2], [3], they typically rely on hand-engineered heuristics based on the geometry of the environment, e.g. optimizing for volumetric gain. Such hand-engineered features do not incorporate the rich semantics and contextual cues of an environment [4].

Given the problem of exploring a previously unknown environment, we develop a framework, Contextual Exploration (Cog Explore), that leverages foundation models, or large language models (LLMs) with natural-language-based representations of the environment to perform navigation using both geometric and contextually rich features, as well as temporal states.

To effectively leverage natural-language, we must ground language utterances to the physical world in which the robot operates. Various approaches have been used to associate language with the physical domain, ranging from probabilistic graph-based structures [5], [6], [7] to end-to-end learning-based methods [8]. We represent the grounding problem as a probabilistic set of planning points, object points, and language priors obtained from a set of vision models and navigation models. Our framework then selects the next best location to explore given this set of priors and a goal. Specifically, we leverage foundation models [9] [10] which have shown the ability to exhibit remarkable contextualization and reasoning by utilizing efficient next token predication.

Navigation solutions powered by LLMs offer distinct advantages over metric-based methods such as frontier point finding [11]. For instance, a robot operating in a school instructed to "Go find a whiteboard eraser" would spend equal time exploring a bathroom or a classroom when using a frontier based planner. However, by integrating an LLM's world knowledge into search strategies, the resulting planning method can leverage the fact that erasers are for more likely to be found in a classroom. Moreover, we can rely on other cues in the environment to influence the search behavior. For example, if a whiteboard is present, the robot should be more exploitative than explorative. This adaptive form of inference, aiming for the most plausible explanation or, casual inference allows CogExplore to create a form of working memory.

LLMs have shown remarkable capabilities in various domains, yet they do not explicitly encode a time dimension, meaning their appreciation of time is implicit [12]. Specifically, LLMs encode memories to be used in future reasoning implicitly as an autoregressive relation instead of an explicitly modeled recurrent relation [13]. We introduce a method to simulate working memory by compressing prior states and justifications into a concise log. By leveraging temporal and contextual reasoning, CogExplore guides robots to navigate diverse environments, while adapting the exploration strategy as new information is acquired.

Our method presents possibilities for advances in scenarios where environments are semantically rich, but of which very little is known a priori. Such scenarios might include diaster relief and search and rescue operations. We validate CogExplore's exploration capabilities with 210 simulations of up to 45 minutes each operating in 3 (Figure 2) different environments across 7 different tasks and show the method is capable of performing temporal, geometric, and semantic reasoning.

II. Related Works

A. Robotic Exploration

Graph-based approaches have proven effective in robotic exploration [5], [6], [7], [14], [15], [16], [17], [11]. These

This work was supported by USDA-NIFA Award Number 2021-67021-3345

Harel Biggie is with Robust Robotics Group, Massachusetts Institute of Technology, Cambridge, MA, USA (email: harelb@mit.edu)

Patrick Cooper is at the University of Colorado - Boulder, Boulder, CO, USA (email: patrick.cooper@colorado.edu

Doncey Albin, Miles Mena, Christoffer Heckman are with the Autonomous Robotics and Perception Group, University of Colorado - Boulder, Boulder, CO, USA (email: doncey.albin@colorado.edu, miles.mena@colorado.edu, christoffer.heckman@colorado.edu

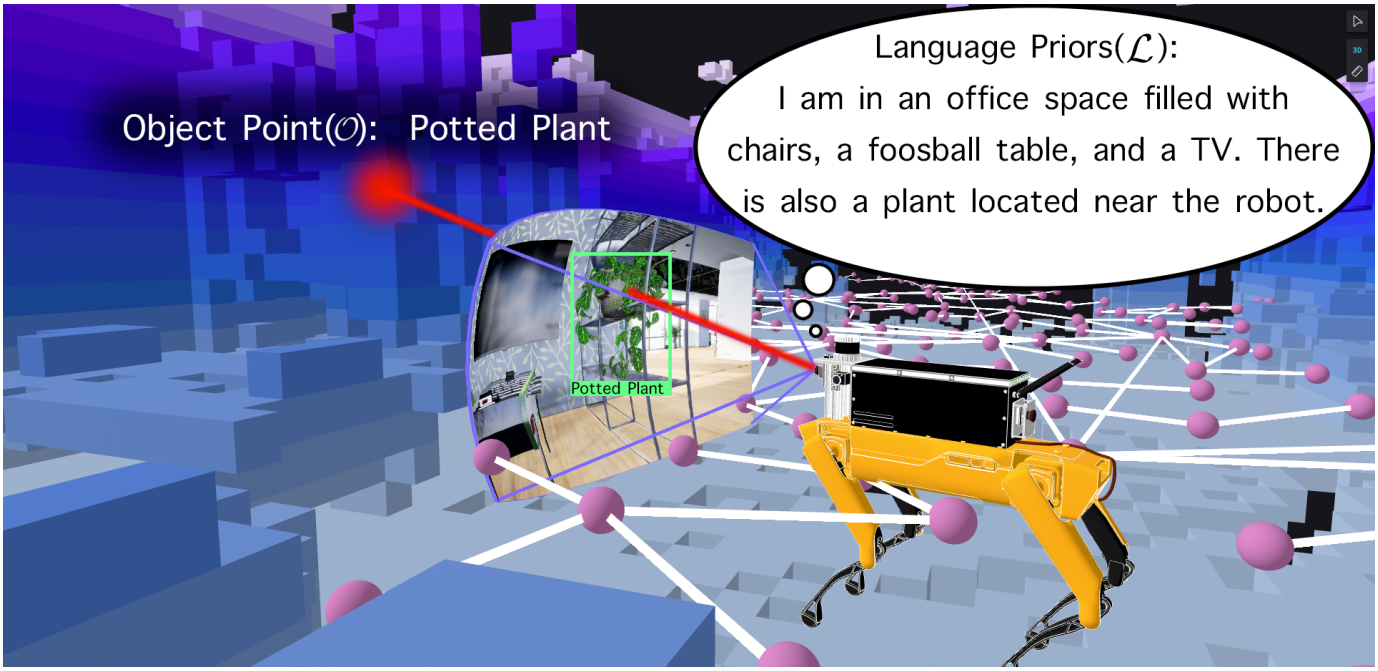


Fig. 1: Spot characterizing its environment through its VQA model (Language Priors), searching for specific objects with its object detection model and creating projections (Object Points shown in red) surrounded by a set of navigable graph points (shown in purple).

methods construct topological representations of the environment to guide exploration and planning. Recent works have incorporated semantics into these graph-based frameworks to enhance performance [18].

In addition to graph-based techniques, neural networks have been applied to robotic navigation, often serving as heuristics [19]. Leveraging advancements in computer vision, modern perception models have been integrated with traditional exploration approaches, leading to improved capabilities [20], [21].

B. LLM-Powered Robotic Navigation

LLMs have emerged as powerful tools for robotic navigation and manipulation. By encoding scene dynamics and task specifications in natural language, LLMs enable robots to reason about their environment and objectives at a high level [22], [23].

LLMs can act as agents capable of decomposing and executing complex tasks through modular prompting [24], [25], [26]. They have demonstrated proficiency in code generation for robotic applications [27], [28], [29], [30], [4]. The combination of embodied agents for low-level skills and LLMs for high-level reasoning has shown promise [23].

End-to-end neural network approaches such as LATTE [31] and CLIPort [32] map natural language intentions to robot actions. HULC [33] combines language conditioning and semantic knowledge for efficient imitation learning. While this approach works well in specific domains, it is not able to learn from the voluminous unsupervised datasets ubiquitous to LLMs.

Grounding natural language instructions to the physical environment is crucial for LLM-powered robotic navigation. Neuro-symbolic approaches [34] and learned traversability functions [35] have been explored for this purpose. PromptCraft [36] tackles the challenge of describing complex navigation tasks through prompt engineering. ORION [37] demonstrates personalized, interactive object navigation using natural language.

While LLMs excel at reasoning over unstructured data [38], [39], [40], [41] using transformer architectures [42], guiding them to desired outcomes and maintaining temporal coherence remains an open problem. Recent works integrating LLMs with object scene representation transformers for task and motion planning [23], [8], [30] still lack the interpretability and guarantees of traditional methods [15].

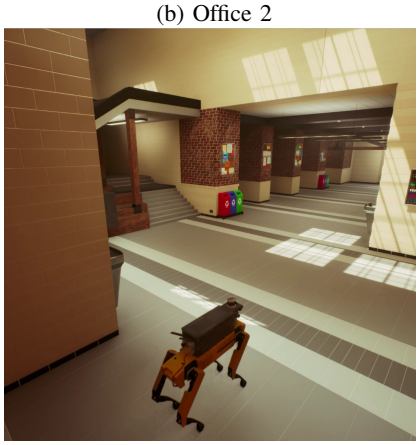
The semantic reasoning abilities of LLMs have been leveraged to guide exploration and planning in novel environments [43], [44]. LLMs have also been utilized for semantic grounding in complex outdoor environments using contextually relevant instructions [44].

LLMs can operate as high-level and low-level planners. In [45], an LLM generates a hierarchy of high-level plans with sequences of sub-goals, and [46] makes direct navigational decisions for frontier selection with an LLM.

Our method introduces new capabilities to this body of work, allowing LLMs to selectively encode relevant states based on environment descriptions and then reason directly over these natural language encoded states, along with temporal states derived from a log of past movements. Informed by this conext, an LLM interaces with the low level planner to direct the robot for semantic exploration.

(a) Office 1

(b) Office 2



(c) School

Fig. 2: Renderings from Unreal Engine Environments

III. CogExplore

Our proposed exploration framework, CogExplore, represents relevant features of the environment as natural language strings. Formally, we represent possible states for the robot as, $\mathcal{S} = \{s_1, s_2, ..., s_n\}$ where s_i is a possible state for the robot. At each state the robot has a set of planning graph points $\mathcal{G} = \{g_1, g_2, ..., g_k\}$, a set of object points $\mathcal{O} = \{o_1, o_2, ..., o_m\}$ and a set of language priors $\mathcal{L}$. Graph points represent traversable areas of the environment, and we represent them as a list in text from $\mathcal{G}_i = \{x, y, z\}$. Object points are represented similarly but with a corresponding label and probability ($\mathcal{O}_i = \{x, y, z, c, p\}$ where c is the class and p is the confidence represented as a probability), obtained from an open vocabulary object detector. Specifically, we utilize YoloWorld [47] and Segment Anything [48] to project open vocabulary detections into 3D points. Examples of $\mathcal{G}, \mathcal{L}$, and $\mathcal{O}$ can be seen in Figure 1 and full details of, the open vocabulary 3D detection system can be found in Section **??** of the Appendix. The language priors, $\mathcal{L}$, consist of three components; environmental descriptions, prior robot states, and the justification for choosing the next state. Environment descriptions $\mathcal{D}$ are obtained from a series of questions generated by foundation model and answered using a multimodal visual question and answering model (VQA) called LLaVA -1.5 [49], as shown in Figure 3. Prior states (s_{i-n}) and the justification for choosing the state ($\mathcal{J}_{i-n}$) and the full set of language priors is $\mathcal{L} = \{\mathcal{J}_{i-n}, s_{i-n}, \mathcal{D}\}$ along with the model's justification for selecting the state. In order to select the next best state, we can model the task as a maximum likelihood estimate:

$$\arg \max_{s_1, s_2, ..., s_t} P(s_g | s_t, \mathcal{S}, \mathcal{O}, \mathcal{G}, \mathcal{L}) \prod_{i=1}^{t} P(s_i | s_{i-1}, \mathcal{S}, \mathcal{O}, \mathcal{G}, \mathcal{L}) \tag{1}$$

At each iteration, the foundation model is asked to direct the robot to the state that is most likely to find the goal state, s_g given the robot's past states and any new observations. The process repeats until the robot arrives at s_g.

A. Exploration Planner

CogExplore's autonomous navigation uses a state-of-the-art graph based exploration planner with frontier point finding [50] to help the language model select reliable states. Realtime mapping is performed using the a voxel based map [51]. The planner was one of the top performing exploration method [52] at the DARPA SubT [53] challenge aimed at advancing exploration capabilities. At each planning iteration, a set of sparse global graph points and dense local points are generated. We employ a 2D Gaussian function to sparsity the distribution of these points, where weights are assigned based on their proximity to the robot's current location. The result is a distribution of low density points in distant areas with a high concentration of points near the agent allowing CogExplore to plan with high granularity locally while still having the ability to explore regions of interest that are further away. The total sets of points is encoded into $\mathcal{G}$.

B. Foundation Model Prompting Schemes

We rely on language foundation models to perform four tasks: Generating questions about the environment, generating object labels, compressing state information, and performing state selection. Foundation models are highly sensitive to the particular presentation of the underlying information [54]. To assist the model's geometric reasoning, we label some of the graph points ($\mathcal{G}$) as new, if they are in an unexplored region. A point is labeled as new if it is beyond a certain distance threshold from all existing points in the graph. This distance

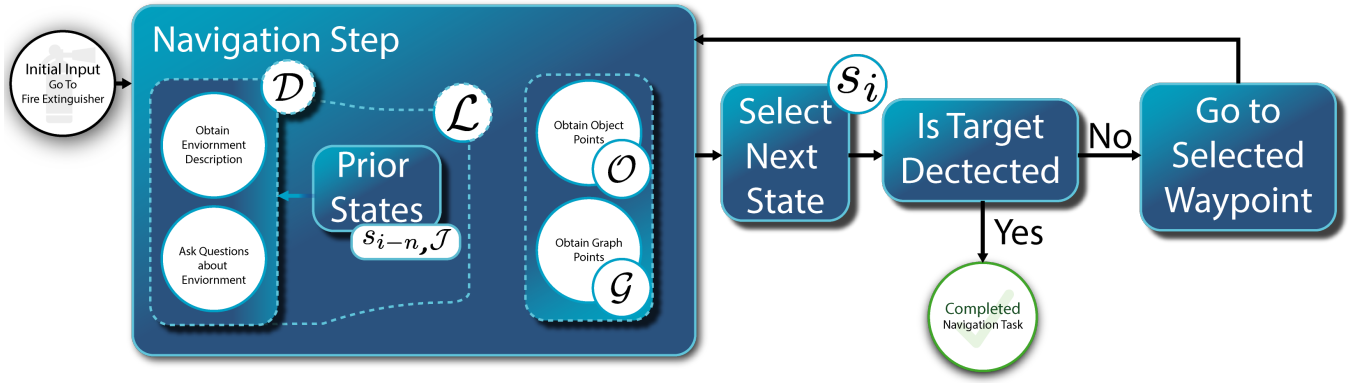


Fig. 3: CogExplore System Diagarm

threshold is determined using a KD-Tree [55] structure to efficiently compute the nearest neighbors.

All of our prompts are highly modular in construction allowing us to incorporate custom visual grounding information ($\mathcal{O}, \mathcal{L}$), prior states s_{i-n}, navigation points ($\mathcal{G}$), and specific instructions designed to handle the strengths and weaknesses of a particular model. In practice, this accommodated the willingness of GPT to select distant points with a high potential for novelty relative to the systematic strategy adopted by Claude. Our prompts were kept consistent for all experiments.

For each iteration of the exploration cycle, the LLM is given a form to fill out, which creates a set of fields for the next waypoint output, as well as fields for a characterization of the environment and a justification field to insert reasoning for the selection of the particular waypoint it chose. All of our prompts follow this mad lib style form:

```
I need to select a waypoint from a
numbered list of graph points, frontier
points and object points. The point I have
 selected is point number: [insert the
graph, frontier or object point numeral
number here]. I am selecting this point
because I believe it makes strategic sense
 to get me closer to solving my navigation
 task "*INSERT_QUERY_HERE*". My
environment can be described as [describe
the environment]. My reasoning is that
this point [insert your reasoning here].
```

Once the generation is complete, the LLM is queried again to compress the prior output into a concise state of 50-100 words ($\mathcal{J}$) these compressed states with justifications enable CogExplore to reason over its past states, facilitating a contiguous exploration process. Each of these logs is appended to a memory window with some fixed length, in practice we discovered a length of 10 was appropriate for both GPT-3.5-Turbo and Claude Haiku.

IV. Experimental Results

A. Experimental Setup

We evaluate the performance of CogExplore's ability explore and find a target object using a simulated Boston Dynamics Spot in three different environments using Unreal Engine [56]. We simulate three RGB cameras (left, right, front) and a 64 beam lidar modeled after an ouster to ensure the simulation is as close to the real-world as possible. Unreal Engine was chosen because the photo-realistic game engine works with object detectors trained on real-world data such as YOLO-World [47]. We compare CogExplore against a A^* direct path metric and the Vision-Enabled Frontier Exploration Planner (VEFEP) described below.

Two office scenes (Office 1, Office 2) and a more complex school environment (School) were designed, which provided the structure for a total of 7 object-retrieval scenarios to be tested, as shown in Table I. In Office 1, there were three tasks involving locating a fire extinguisher in different locations, denoted as FE1, FE2, and FE3. Office 2 featured two tasks: finding a coffee table (CT) and an office chair (OC). In School, the tasks were to locate a whiteboard (WB) and a bookshelf (BS).

For each task, we simulate the navigation capabilities of a Boston Dynamics Spot and allow the robot to explore for 45 minutes. A run terminates once the target object is found. We conduct 15 trials per task on Azure "NC64as_T4_v3" instances with 4 Nvidia T4 GPUs. We utilize 3 GPUs per simulation, one for the unreal simulator, one for the object detector, and one for the VQA model. We note that for Office 1, and the school scene, the simulator runs in real-time. While in Office 2 the simulator runs in half-time due to higher quality graphical assets. While CogExplore does run in real-time, with the assistance of cloud computing we find runtimes are not an accurate measurement of exploration performance due to uncontrollable factors such latency and API call throttling from leading foundation model providers.

We compare the performance of CogExplore running with Anthropic's *claude-3-haiku-20240307* (CE-H) and OpenAI'S *gpt-3.5-turbo-0125* (CE-3.5) as the backend models to a baseline Vision-Enabled Frontier Exploration Planner (VEFEP).

Office 1 (572 m^2): Figure 2a			Office 2 (1450 m^2): Figure 2b			School (1287 m^2): Figure 2c		
Task	**Direct Path (m)**	**VEFEP # Timeout**	**Task**	**Direct Path (m)**	**VEFEP # Timeout**	**Task**	**Direct Path (m)**	**VEFEP # Timeout**
Fire Extinguisher 1 (FE1)	25.2	0	Office Chair (OC)	33.7	3	Whiteboard Eraser (WE)	26.1	5
Fire Extinguisher 2 (FE2)	27.1	0	Coffee Table (CT)	26.2	6	Book-shelf (BS)	13	2
Fire Extinguisher 3 (FE3)	15.6	0						

TABLE I: Simulation Environments and Corresponding Tasks. Each run was given 45 minutes of simulation time to complete. None of the CogExplore runs timed out, and the VEFEP timeouts are shown here.

VEFEP calculates the exploration gain heuristic g for a given path σ_i by summing the volumetric gain $\mathbf{VG}$ weighted by a function for each vertex in the path [57]. Parameters are based on the ones used by the authors in [50]. This planner explores until it detects the target object using the same 3D open Vocabulary detector used by CogExplore and described in section III. All constructed environments, simulation code, and CogExplore code will be released upon publication.

Quantitative Results: From Figure 4 we can see that in Office 1, the smallest environment, VEFEP, CE-H, and CE-3.5 all have similar mean path lengths. As the environment size increases in Office 2, making the exploration tasks more complex, we observe that on the office chair task both CE-H and CE-3.5 outperform VEFEP. We also note that VEFEP, timed out in 3 out of 15 of the runs as noted in Table I. VEFEP had the lowest median run length in the coffee table task. However, it also failed to find the table in 6 out of 15 experiments, as shown in Table I. A similar result is present in the school environment with the bookshelf task where CE-H outperforms VEFEP on path length but CE-3.5 does not. However, both CE methods complete the task 100 percent of the time and VEFEP timed out in 2 out of 15 tasks. In the whiteboard eraser task, both CE models significantly outperform VEFEP, which also had 5 out of 15 failures or a 33.3% failure rate. Of note, is that *none* of the CE methods failed at the exploration task in any of the environments.

Qualitative Results: In Figure 6 we observe that in the Whiteboard Eraser task CE-H and CE-3.5 only enter the room with the eraser once whereas VEFEP enters, exits and proceeds to loop around the environment. Similarly, in the Go to the Office Chair task CE-H and CE-3.5, immediately go to the chair and take more efficient paths to get there. In contrast, VEFEP heads outside and explores the areas significantly before returning to indoors to find the chair.

We provide anecdotal examples of the model's reasoning in Figure 5, where we see examples of the model performing spatial reasoning (left), semantic reasoning (center) and temporal reasoning (right). At each iteration we can probe why the model made a decision which highlights the explainability of the CogExplore method.

V. Discussion

Our results highlight the robustness of CogExplore's exploration abilities with a *100% success rate* across all environments with both GPT and Haiku variants, in contrast to VEFEP's 15.2% overall failure rate and 40% failure rate at the coffee table task. Moreover, CogExplore's use of natural language justifications for each state selection allow us to directly probe why the framework is acting in a certain manner and understand the rationale behind the more efficient exploration paths, increasing explainability. From these justifications, we can see that the model is capable of performing geometric reasoning, semantic reasoning, temporal reasoning, and fault-tolerant reasoning.

Cog Explore's ability to leverage a variety of reasoning mechanisms through encoding environmental features into natural language strings enables the sharp performance gains from heuristic-based methods like VEFEP. Frontier-based exploration methods such as VEFEP rely on hand tuning of the volumetric gain weights to influence the exploration behavior, whereas CogExplore adapts based on observations. This is highlighted in the example where our framework tells the robot to choose a further state because no clues were found indicating that the fire extinguisher is nearby:

```
"The point is far enough to explore a
different vantage point within the
domestic setting as no indications of the
fire extinguisher are present."
```

Geometric Reasoning. We see geometric reasoning on the right side of Figure 5 where the model explicitly chooses a point behind the robot. It is aware that the robot can not detect objects directly behind it, since there is no rear camera. This variety of intuitive geometric reasoning directly contributes to the shorter exploration path lengths demonstrated by CogExplore.

Semantic Reasoning. Another key factor that contributes to robust and efficient performance is the model's ability to semantically reason over cues in the environment. In the center of Figure 5 we see the model choosing a nearby point because a desk was found which could contain a white board eraser (the desired object in the exploration task). Both objects are commonly located in classroom and office settings, hence their shared semantic similarity. Traditional frontier-based planners continue to explore other areas based on the original heuristic for volumetric gain despite these cues, which is also evident by the path taken by VEFEP in Figure 6a.

Temporal Reasoning. Beyond reasoning over the current state, efficient path exploration requires an agent to reason over its past states. In CogExplore's case, we note the robot

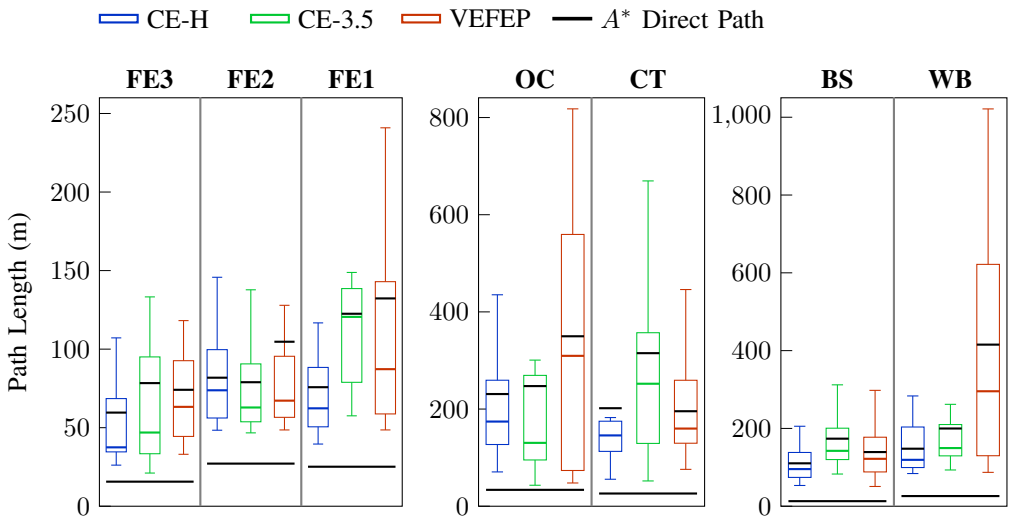


Fig. 4: Path length comparisons for each method (CE-3.5, CE-H, VEFEP) on completing each of the seven tasks. Black line for each whisker plot is the mean.

venturing to a new area despite having a point labeled as a fire extinguisher (the target exploration object). The robot already explored the nearby area in detail in a prior state, and the target object was never reached. The model correctly concludes the detection was in error. CogExplore justifies this change based on an understanding that object detections can be fallacious, which also highlights the framework's ability to perform fault-tolerant reasoning and remain robust to noisy sensor observations. In contrast, it was observed that the VEFEP planner would frequently stick around objects as the location of the 3D object projections were refined (as new observations were obtained), despite this not being an effective strategy after a few iterations.

We can also see from Figure 6b that VEFEP suffered from exploring near the office chair but not actually reaching it. This is attributed to obstructed projections in the object detection system. Since the system performs a ray cast, obstructed views can cause object projections to fall short of the actual location. If no new observations update this position, then VEFEP will veer away from the area, whereas CogExplore will usually probe the area or come back to it if other cues indicate it's a good area to keep exploring.

VI. Conclusions and Limitations

Grounding vision models within simulated imagery poses challenges due to the limitations of VQA and object detection models, which can produce hallucinations, false negatives, and false positives. Our method addresses these errors by maintaining continuity in environment descriptions, limiting the number of target objects generated by the LLM, and requiring the LLM to interpret the environment across multiple generations. However, our approach lacks the ability to autonomously determine the correctness of identified target objects without ongoing experimentation and relies on simulation for confirmation or rejection, necessitating supervisor evaluation in potential deployments. Furthermore, while our discussion of results assumes the effectiveness of our approach based on its demonstrated performance and the compatibility of its justifications with the evaluated environments and objectives, we acknowledge the possibility of post hoc reasoning introducing a potential source of error.

CogExplore is a comprehensive framework for leveraging foundation models in robotic exploration tasks. The framework offers the ability to perform exploration that is geometrically, semantically, and temporally aware while remaining resilient to failures in grounding. Performance is evaluated across 210

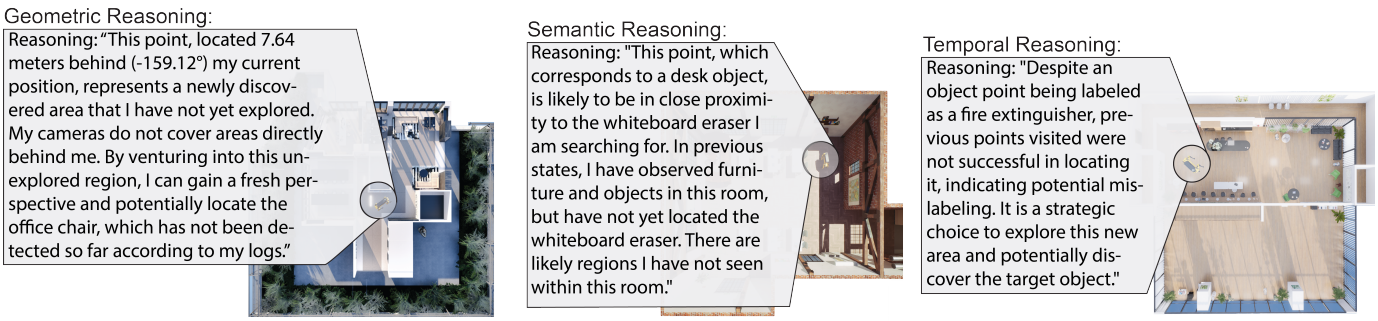
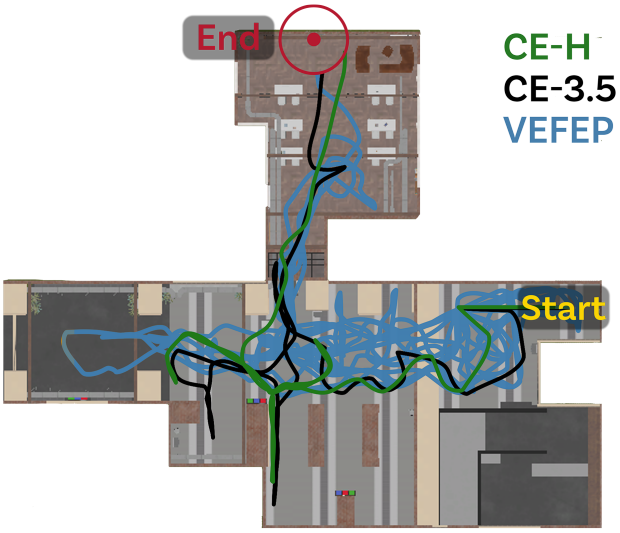
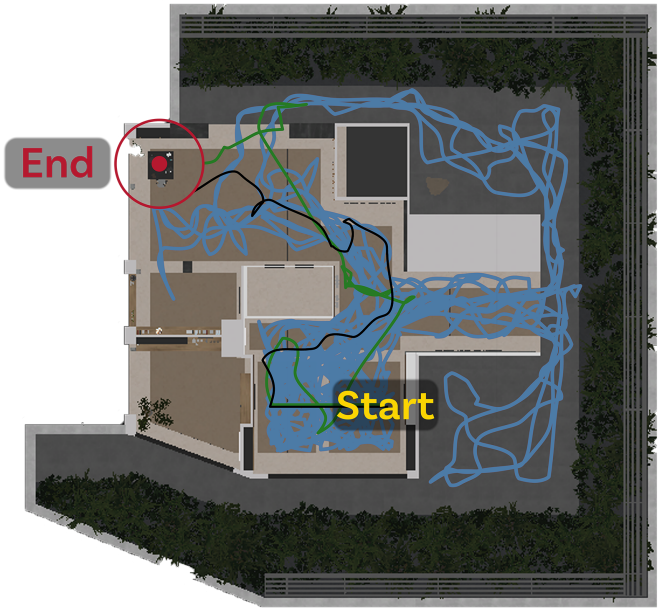


Fig. 5: Example Runs Demonstrating Varieties of Reasoning



(a) Note that the VEFEP re-visits the same places several times before eventually locating the goal object, whereas the CogExplore is able to more effectively reason over unexplored regions.



(b) Note that VEFEP obtains noisy projections and leaves the area rather than continuing to search the area for the office chair. Whereas Cog Explore is able to reason over the noisy projections.

Fig. 6: Bird's Eye Views: Go to the Whiteboard Eraser, School (a) and Go to the Office Chair, Office 2 (b)

photorealistic simulations in 3 different environments with 7 different exploration tasks. Our findings reveal that as the complexity of the exploration task increases, in terms of environment size and trajectory length, CogExplore's performance advantage over a vision-enabled exploration planner becomes more pronounced. This positive correlation shows that CogExplore is particularly well-suited for handling challenging navigation scenarios.

References

[1] H. Biggie, E. R. Rush, D. G. Riley, S. Ahmad, M. T. Ohradzansky, K. Harlow, M. J. Miles, D. Torres, S. McGuire, E. W. Frew *et al.*, "Flexible supervised autonomy for exploration in subterranean environments," *Field Robotics*, vol. 3, pp. 125–189, 2023.

[2] M. Tranzatto, T. Miki, M. Dharmadhikari, L. Bernreiter, M. Kulkarni, F. Mascarich, O. Andersson, S. Khattak, M. Hutter, R. Siegwart *et al.*, "Cerberus in the darpa subterranean challenge," *Science Robotics*, vol. 7, no. 66, p. eabp9742, 2022.

[3] A. Agha, K. Otsu, B. Morrell, D. D. Fan, R. Thakker, A. Santamaria-Navarro, S.-K. Kim, A. Bouman, X. Lei, J. Edlund *et al.*, "Nebula: Quest for robotic autonomy in challenging environments; team costar at the darpa subterranean challenge," *arXiv preprint arXiv:2103.11470*, 2021.

[4] H. Biggie, A. N. Mopidevi, D. Woods, and C. Heckman, "Tell Me Where to Go: A Composable Framework for Context-Aware Embodied Robot Navigation," in *Conference on Robot Learning*. PMLR, 11 2023. [Online]. Available: https://proceedings.mlr.press/v229/biggie23a/biggie23a.pdf

[5] S. Tellex, T. Kollar, S. Dickerson, M. Walter, A. Banerjee, S. Teller, and N. Roy, "Understanding natural language commands for robotic navigation and mobile manipulation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 25, 2011, pp. 1507–1514.

[6] T. M. Howard, S. Tellex, and N. Roy, "A natural language planner interface for mobile manipulators," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2014, pp. 6652–6659.

[7] T. Kollar, S. Tellex, D. Roy, and N. Roy, "Toward understanding natural language directions," in *2010 5th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. IEEE, 2010, pp. 259–266.

[8] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu *et al.*, "Palm-e: An embodied multimodal language model," *arXiv preprint arXiv:2303.03378*, 2023.

[9] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *CoRR*, vol. abs/2005.14165, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[10] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. R. Lavaud, L. Saulnier, M.-A. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. L. Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mixtral of experts," 2024.

[11] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA'97.'Towards New Computational Principles for Robotics and Automation'*. IEEE, 1997, pp. 146–151.

[12] F. Deng, J. Park, and S. Ahn, "Facing off world model backbones: Rnns, transformers, and s4," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *CoRR*, vol. abs/1706.03762, 2017. [Online]. Available: http://arxiv.org/abs/1706.03762

[14] S. M. LaValle *et al.*, "Rapidly-exploring random trees: A new tool for path planning," 1998.

[15] I. Noreen, A. Khan, and Z. Habib, "Optimal path planning using rrt* based approaches: a survey and future directions," *International Journal of Advanced Computer Science and Applications*, vol. 7, no. 11, 2016.

[16] S. Karaman, M. R. Walter, A. Perez, E. Frazzoli, and S. Teller, "Anytime motion planning using the rrt," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 1478–1483.

[17] X. Lan and S. Di Cairano, "Continuous curvature path planning for semi-autonomous vehicle maneuvers using rrt," in *2015 European control conference (ECC)*. IEEE, 2015, pp. 2360–2365.

[18] J. Crespo, J. C. Castillo, O. M. Mozos, and R. Barber, "Semantic information for robot navigation: A survey," *Applied Sciences*, vol. 10, no. 2, p. 497, 2020.

[19] T. T. Mac, C. Copot, D. T. Tran, and R. De Keyser, "Heuristic approaches in robot path planning: A survey," *Robotics and Autonomous Systems*, vol. 86, pp. 13–28, 2016.

[20] S. Y. Gadre, M. Wortsman, G. Ilharco, L. Schmidt, and S. Song, "Clip on wheels: Zero-shot object navigation as object localization and exploration," *arXiv preprint arXiv:2203.10421*, 2022.

[21] C. Huang, O. Mees, A. Zeng, and W. Burgard, "Visual language maps for robot navigation," *arXiv preprint arXiv:2210.05714*, 2022.

[22] F. Zeng, W. Gan, Y. Wang, N. Liu, and P. S. Yu, "Large language models for robotics: A survey," 2023.

[23] A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," in *Conference on Robot Learning*. PMLR, 2023, pp. 287–318.

[24] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.

[25] T. Gupta and A. Kembhavi, "Visual programming: Compositional visual reasoning without training," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 14 953–14 962.

[26] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," 2022.

[27] A. Madaan, S. Zhou, U. Alon, Y. Yang, and G. Neubig, "Language models of code are few-shot commonsense learners," *arXiv preprint arXiv:2210.07128*, 2022.

[28] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Chi conference on human factors in computing systems extended abstracts*, 2022, pp. 1–7.

[29] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022.

[30] O. Saycan, S. Garg, E. Meyerson, J. Tsai, V. Ordonez, R. Mottaghi, and A. Farhadi, "Progprompt: Generating situated robot task plans using large language models," 2022.

[31] A. Bucker, L. Figueredo, S. Haddadin, A. Kapoor, S.-Y. Ma, S. Vemprala, and R. Bonatti, "Latte: Language trajectory transformer," 2022.

[32] M. Shridhar, L. Manuelli, and D. Fox, "Cliport: What and where pathways for robotic manipulation," in *Proceedings of the 5th Conference on Robot Learning (CoRL)*, 2021.

[33] O. Mees, L. Hermann, and W. Burgard, "What matters in language conditioned robotic imitation learning over unstructured data," *IEEE Robotics and Automation Letters (RA-L)*, vol. 7, no. 4, pp. 11 205–11 212, 2022.

[34] J. Mao, C. Gan, P. Kohli, J. B. Tenenbaum, and J. Wu, "The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision," *arXiv preprint arXiv:1904.12584*, 2019.

[35] D. Shah, B. Eysenbach, G. Kahn, N. Rhinehart, and S. Levine, "Ving: Learning open-world navigation with visual goals," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 13 215–13 222.

[36] S. Vemprala, R. Bonatti, A. Bucker, and A. Kapoor, "Chatgpt for robotics: Design principles and model abilities," Microsoft, Tech. Rep. MSR-TR-2023-8, February 2023. [Online]. Available: https://www.microsoft.com/en-us/research/publication/chatgpt-for-robotics-design-principles-and-model-abilities/

[37] Y. Dai, R. Peng, S. Li, and J. Chai, "Think, act, and ask: Open-world interactive personalized robot navigation," *ICRA*, 2024.

[38] L. Floridi and M. Chiriatti, "Gpt-3: Its nature, scope, limits, and consequences," *Minds and Machines*, vol. 30, pp. 681–694, 2020.

[39] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[40] S. Tellex, N. Gopalan, H. Kress-Gazit, and C. Matuszek, "Robots that use language," in *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 3. Annual Reviews, 2020, pp. 25–55.

[41] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[42] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[43] D. Shah, M. R. Equi, B. Osiński, F. Xia, B. Ichter, and S. Levine, "Navigation with large language models: Semantic guesswork as a heuristic for planning," in *Conference on Robot Learning*. PMLR, 2023, pp. 2683–2699.

[44] D. Shah, A. Malhotra, S. Singh, and A. Gupta, "Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action," 2021.

[45] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, "Llm-planner: Few-shot grounded planning for embodied agents with large language models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 2998–3009.

[46] S. Yuan, H. U. Unlu, H. Huang, C. Wen, A. Tzes, and Y. Fang, "Exploring the reliability of foundation model-based frontier selection in zero-shot object goal navigation," in *International Conference on Pattern Recognition*. Springer, 2025, pp. 119–134.

[47] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, "Yolo-world: Real-time open-vocabulary object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.

[48] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo *et al.*, "Segment anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 4015–4026.

[49] H. Liu, C. Li, Y. Li, B. Li, Y. Zhang, S. Shen, and Y. J. Lee, "Llava-next: Improved reasoning, ocr, and world knowledge," January 2024. [Online]. Available: https://llava-vl.github.io/blog/2024-01-30-llava-next/

[50] T. Dang, M. Tranzatto, S. Khattak, F. Mascarich, K. Alexis, and M. Hutter, "Graph-based subterranean exploration path planning using aerial and legged robots," *Journal of Field Robotics*, 10 2020.

[51] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3d mapping framework based on octrees," *Autonomous robots*, vol. 34, pp. 189–206, 2013.

[52] M. Tranzatto, M. Dharmadhikari, L. Bernreiter, M. Camurri, S. Khattak, F. Mascarich, P. Pfreundschuh, D. Wisth, S. Zimmermann, M. Kulkarni, V. Reijgwart, B. Casseau, T. Homberger, P. D. Petris, L. Ott, W. Tubby, G. Waibel, H. Nguyen, C. Cadena, R. Buchanan, L. Wellhausen, N. Khedekar, O. Andersson, L. Zhang, T. Miki, T. Dang, M. Mattamala, M. Montenegro, K. Meyer, X. Wu, A. Briod, M. Mueller, M. Fallon, R. Siegwart, M. Hutter, and K. Alexis, "Team cerberus wins the darpa subterranean challenge: Technical overview and lessons learned," 2022.

[53] DARPA, "DARPA Subterranean Challenge," 2022, https://www.darpa.mil/program/darpa-subterranean-challenge. [Online]. Available: https://www.darpa.mil/program/darpa-subterranean-challenge

[54] Y. Chen, R. Zhong, N. Ri, C. Zhao, H. He, J. Steinhardt, Z. Yu, and K. McKeown, "Do models explain themselves? counterfactual simulatability of natural language explanations," 2023.

[55] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.

[56] Epic Games, "Unreal engine." [Online]. Available: https://www.unrealengine.com

[57] A. Bachrach, "Trajectory bundle estimation for perception-driven planning," 2013. [Online]. Available: https://api.semanticscholar.org/CorpusID:36535521